

INFO 09 : Dichotomie et Méthode de Newton

Lycée Alain-Fournier

01 Février 2021

1 I) Dichotomie

2 II) Méthode de Newton

I) Dichotomie 1) Principe de la méthode

Principe Théorique :

On suppose qu'une fonction continue sur un intervalle $[a, b]$ prends des valeurs de signes opposés. Le théorème des valeurs intermédiaires assure que f s'annule au moins une fois dans cet intervalle.

Il y a deux démonstrations :

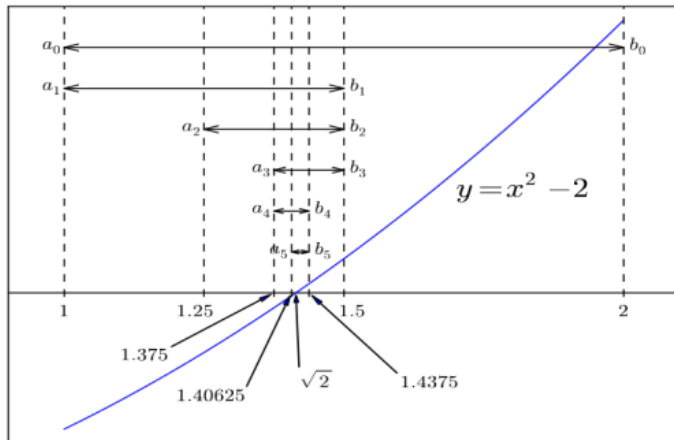
La "jolie" : considérer la borne supérieure de l'ensemble des $x \in [a, b]$ tels que $f(x)$ soit du signe de $f(a)$.

La technique : qui utilise deux suites adjacentes et le principe de dichotomie. Cette démonstration donne un moyen simple d'approcher le point où f s'annule.

On construit deux segments emboîtés de plus en plus petits s'approchant d'un singleton $\{l\}$, l vérifiant $f(l) = 0$ par continuité. Le plus important est de maintenir l'invariant :

f change de signe entre les deux extrémités du segment

On l'illustre sur la figure suivante



2) Algorithme

On traduit le principe de dichotomie en Python :

```
def dichotomie(f,a,b,eps):  
    assert f(a)*f(b) <=0 and eps >0 # Pour s'assurer  
    c,d=a,b                          # qu'il y a une solution  
    fc,fd = f(c),f(d)                # On stocke pour ne pas recalculer  
  
    while d - c > 2*eps:  
        m=(c+d)/2  
        fm=f(m)  
        if fc*fm <= 0 :             # Solution sur [c,m]  
            d,fd = m,fm  
        else:                        # Solution sur [m,d]  
            c,fc = m,fm  
    return((c+d)/2)                  # On va en parler
```

Et ça fonctionne :

```
>>> dichotomie(lambda x : x**2 -2,1,2,10**-6)
1.4142141342163086
>>> 2**0.5
1.4142135623730951
```

Pour la sortie il y a généralement deux choix :

- ▷ Le milieu $(c+d)/2$ du dernier segment : on se dit que bon ça devrait être par là.
- ▷ Renvoyer l'extrémité dont l'image par f est la plus proche de zéro : on fait un appel supplémentaire à f mais c'est réputé un peu plus stable

3) L'arsenal habituel

On considère c_k et d_k les valeurs des variables c et c au k ième passage en boucle.

Terminaison

La terminaison se démontre avec l'invariant : $d_k - c_k = \frac{b-a}{2^k}$.
Pour k grand on aura bien $d_k - c_k < \epsilon$ donc la boucle s'arrête.

Correction

La correction se démontre avec l'invariant : $f(c_k)f(d_k) \leq 0$.
C'est un simple raisonnement mathématique similaire à la démonstration du TVI.

Complexité

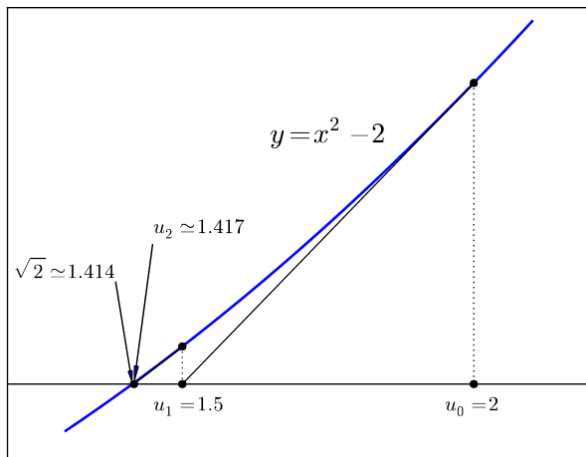
On s'intéresse au nombre d'appel à f , où les calculs internes peuvent être gourmands. De $d_k - c_k = \frac{b-a}{2^k} > \epsilon$ on extrait que la boucle est réalisée $\ln_2 \frac{b-a}{\epsilon} - 1$ fois.

II) Méthode de Newton 1) Principe de la méthode

La méthode de Newton sert à approcher un zéro de fonction. Elle consiste, en partant d'un point qu'on espère pas trop éloigné, à prendre le zéro de la tangente à la courbe en ce point, et de réitérer le procédé.

Si on reprend l'exemple de la racine de $f(x) = x^2 - 2$, on choisit la valeur initiale $u_0 = 2$. L'intersection de la tangente à f en u_0 coupe l'axe des abscisses en $u_1 = 1,5$. En suivant la tangente à f en u_1 , on coupe l'axe des abscisses en $u_2 \simeq 1,4167$

On l'illustre sur la figure suivante :



On peut se convaincre que la suite u_n converge vers $\sqrt{2}$.

On obtient facilement que $u_{n+1} = g(u_n)$ où g est définie par $g(x) = \frac{1}{2} \left(x + \frac{2}{x} \right)$, et on peut facilement prouver cette convergence, $\sqrt{2}$ étant un point fixe de g .

La dérivée de g en $a = \sqrt{2}$ est nulle, ce qui permet une vitesse de convergence élevée.

En posant $\delta_n = u_n - \sqrt{2}$ on a $\frac{\delta_{n+1}}{\delta_n} = \frac{g(u_n) - g(a)}{u_n - a} \xrightarrow{n \rightarrow +\infty} g'(a)$.

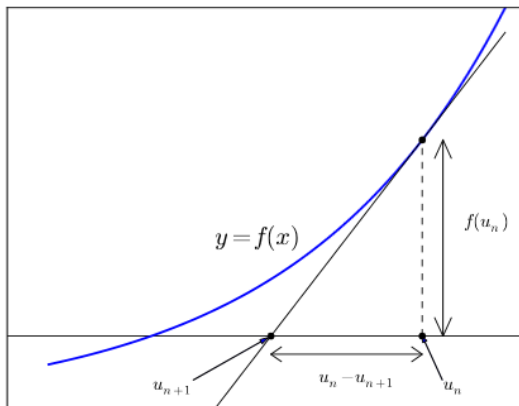
Ainsi plus $|g'(a)|$ est faible, plus la convergence est rapide.

Ici nous avons $\delta_{n+1} \leq \frac{\delta_n^2}{2}$. On dit que la convergence est quadratique.

Donnons-nous une idée, pour quelle valeur de n à-t-on $|\delta_n| \leq \frac{1}{2^{1000}}$?

2) Algorithme

L'algorithme décrit est relativement simple : suivre les tangentes, jusqu'à obtenir une précision suffisante. Il est nécessaire par contre de ne jamais rencontrer de points en lequel la dérivée de f s'annule.



La figure précédente nous permet d'obtenir la relation de récurrence : la pente vaut $\frac{f(u_n)}{u_n - u_{n+1}}$ d'une part, et $f'(u_n)$ de l'autre. On obtient donc $u_{n+1} = u_n - \frac{f(u_n)}{f'(u_n)}$. (On peut l'obtenir aussi via calculs).

La condition d'arrêt n'est pas simple à choisir, contrairement à la dichotomie. On décide, faute de mieux, de s'arrêter quand la différence entre deux termes consécutifs est assez petite. Rien ne permet d'affirmer qu'on obtient bien une valeur approchée d'une racine, mais généralement ça fonctionne bien.

L'algorithme est donc le suivant : c'est un simple calcul de terme d'une suite définie par récurrence avec $u_{n+1} = u_n - \frac{f(u_n)}{f'(u_n)}$.

Données : f, f', u_0, eps

$u = u_0$

$v = u - f(u)/f'(u)$

Tant que $v-u > \text{eps}$ faire

$u = v$

$v = v - f(v)/f'(v)$

Résultat : v

Démontrer la terminaison et la correction de l'algorithme est une plaie :

- On peut rencontrer des divisions par zéro
- La terminaison n'est pas assurée
- Et le résultat renvoyé peut être très loin d'un zéro de f !

Mais si il y a convergence, elle être très rapide.

En pratique les programmes utilisant la méthode de Newton sont plus compliqués, en essayant de contrôler les problèmes mentionnés, grâce à des conditions mathématiques plus ou moins complexes.

Newton ou Dichotomie?

- La dichotomie est qualifiée de *Slow but sure* : simplicité et robustesse. La condition sur la fonction est simple : continuité et changement de signe.
- La méthode de Newton est bien plus rapide, mais un peu fragile. Si on connaît une approximation d'un zéro, on sait où l'on doit arriver.
- Au final, on peut mixer les méthodes : localiser les zéros par dichotomie, puis les approcher par Newton.

3) Évaluation de la dérivée

La méthode de Newton nécessite la connaissance de la dérivée de la fonction. Cette dérivée peut-être connue ou calculée, à la main ou avec un logiciel de calcul formel, mais peut aussi ne pas être accessible, notamment si la fonction est connue via des valeurs et non par une expression.

Dans ce dernier cas, on doit alors approximer les valeurs de f' .

Le théorème de Taylor-Young permet de régler le problème.

A l'ordre 2 on a : $f(x_0 + h) = f(x_0) + hf'(x_0) + \frac{f''(x_0)}{2}h^2 + o(h^2)$

Ce qui nous donne $\frac{f(x_0+h)-f(x_0)}{h} - f'(x_0) \sim \frac{f''(x_0)}{2}h$.

On peut donc faire l'approximation $f'(x) \sim \frac{f(x+h)-f(x)}{h}$ qui sera d'autant plus précise que $f''(x_0)$ soit petit.

On peut même faire encore mieux : $\frac{f(x_0+h)-f(x_0-h)}{2h} - f'(x_0) \sim \frac{f^{(3)}(x_0)}{6}h^2$

Lorsqu'on fait ces approximations, on parle plutôt de méthode de la sécante que de la méthode de Newton.

4) Mise en œuvre

On ne fait pas de test sur la convergence de la méthode, l'utilisateur applique à ses risques et périls.

```
def newton(f,fp,x0,eps):  
    u=x0  
    v=u-f(u)/fp(u)  
    while abs(v-u) > eps:  
        u,v=v, v - f(v)/fp(v)  
    return(v)
```

On peut le tester sur nos classiques

```
newton(math.sin,math.cos,3,0.0001)  
3.141592653589793
```

```
newton(lambda x : x**2-2,lambda x : 2*x,2,10**(-6))  
1.4142135623730951
```

5) Du côté de numpy/scipy

La dichotomie est bien sûr présente :

```
import scipy.optimize as sp
import numpy as np
>>> sp.bisect(np.sin,3,4)
3.141592653589214
```

La méthode de Newton est présente, et contient plusieurs arguments optionnels.

```
>>> sp.newton(np.sin,3,np.cos)
3.1415926535897931
```

Si on ne rentre pas la dérivée, c'est la méthode de la sécante qui est appliquée

```
>>> sp.newton(np.sin,3)
3.1415926535897931
```

Notons la méthode de Halley, qui utilise aussi f'' et qui est cubique.

```
>>> sp.newton(np.sin, 3, np.cos, fprime2=lambda x : -1*np.sin(x))  
3.1415926535897931
```

La méthode de Brent décide elle-même (en gros) quelle méthode utiliser à chaque itération :

```
>>> sp.brentq(np.sin, 3, 4)  
3.141592653589793
```

On peut appliquer la méthode de Newton aux complexes, et obtenir des résultats surprenants.

En l'appliquant à la fonction $f(z) = z^3 - 1$, on observe que selon le point de départ on converge vers une des trois solutions, sauf pour certains. En représentant cela on obtient la Fractale de Newton.

